EXP 2 : Linear Regression

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# User-defined data
X = np.array([[1], [2], [3], [4], [5], [6]])
y = np.array([20, 30, 40, 50, 60, 70])

# Create and train model
model = LinearRegression()
model.fit(X, y)

# Predict values
pred = model.predict(X)
new_prediction = model.predict([[7]])

# Display results
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
print("Predicted marks for 7 hours:", new_prediction[0])
print("MSE:", mean_squared_error(y, pred))
print("R2 Score:", r2_score(y, pred))

# Plot graph
plt.scatter(X, y, label="Actual Data")
plt.plot(X, pred, label="Regression Line")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.legend()
plt.show()
```

Experiment 3 — Logistic Regression

```
# Experiment 3: Logistic Regression classification
# Install if required: python -m pip install scikit-learn

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# Input feature: study hours; target: 0 = fail, 1 = pass
X = [[1], [2], [3], [4], [5], [6], [7], [8]]
y = [0, 0, 0, 0, 1, 1, 1, 1]

# Create and train model
model = LogisticRegression()
model.fit(X, y)

# Predict classes for new students
X_new = [[2], [5], [7]]
predictions = model.predict(X_new)

print("Predictions (0 = fail, 1 = pass):", predictions)
print("Training accuracy:", accuracy_score(y, model.predict(X)))
print("Confusion matrix on training data:")
print(confusion_matrix(y, model.predict(X)))
```

Experiment 8 — Principal Component Analysis (PCA)

```
# Experiment 8: Dimensionality reduction using PCA
# Install if required: python -m pip install scikit-learn matplotlib

import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Sample dataset: 5 observations and 3 features
X = [
    [2, 3, 5],
    [3, 4, 6],
    [4, 5, 7],
    [5, 7, 8],
    [6, 8, 9]
]

# Standardize features before PCA
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Reduce from 3 features to 2 principal components
pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X_scaled)

print("Original data shape:", len(X), "x", len(X[0]))
print("Reduced data shape:", X_reduced.shape)
print("Explained variance ratio:", pca.explained_variance_ratio_)
print("Total variance retained:", sum(pca.explained_variance_ratio_))

# Plot observations in the two-component space
plt.scatter(X_reduced[:, 0], X_reduced[:, 1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA: Data in Two Dimensions")
plt.grid(True)
plt.show()
```

EXP 7 : DBSCAN

```
import numpy as np
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

# User-defined data points
X = np.array([
   [1, 2], [2, 2], [2, 3],
   [8, 8], [8, 9], [9, 8],
   [25, 25]
])

# Create and fit DBSCAN
model = DBSCAN(eps=2, min_samples=2)
labels = model.fit_predict(X)

# Display clusters
print("Cluster Labels:", labels)

# Plot clusters and noise
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.title("DBSCAN Clustering")
plt.show()
```

Experiment 1 — ML Workflow (Decision Tree Classification)

```
# Experiment 1: Basic Machine Learning workflow
# Install if required: python -m pip install scikit-learn

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Input feature: study hours; target: 0 = fail, 1 = pass
X = [[1], [2], [3], [4], [5], [6]]
y = [0, 0, 0, 1, 1, 1]

# Create and train the model
model = DecisionTreeClassifier(random_state=0)
model.fit(X, y)

# Predict the class for a new example
new_student = [[4]]
prediction = model.predict(new_student)

print("Prediction (0 = fail, 1 = pass):", prediction[0])
print("Training accuracy:", accuracy_score(y, model.predict(X)))
```

Experiment 5 — Ensemble Learning using AdaBoost

```
# Experiment 5: AdaBoost classifier
# Install if required: python -m pip install scikit-learn
# For older sklearn versions, use base_estimator=weak instead of estimator=weak.

from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Sample data: one feature and two classes
X = [[1], [2], [3], [4], [5], [6], [7], [8]]
y = [0, 0, 0, 0, 1, 1, 1, 1]

# Decision stump as a weak learner
weak = DecisionTreeClassifier(max_depth=1, random_state=0)

# Create and train AdaBoost
try:
    model = AdaBoostClassifier(estimator=weak, n_estimators=10, random_state=0)
except TypeError:
    # Compatibility with older scikit-learn versions
    model = AdaBoostClassifier(base_estimator=weak, n_estimators=10, random_state=0)

model.fit(X, y)

# Predict new examples
X_new = [[2], [6]]
predictions = model.predict(X_new)

print("Predictions:", predictions)
print("Training accuracy:", accuracy_score(y, model.predict(X)))
```

Experiment 4 — ID3-style Decision Tree using Entropy

```
# Experiment 4: Decision Tree using entropy (ID3 splitting criterion)
# Install if required: python -m pip install scikit-learn matplotlib
# Note: sklearn's entropy-based tree is not a dedicated original ID3 implementation.

from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# Features: Outlook (0=sunny, 1=cloudy), Temperature (0=cool, 1=warm)
X = [
    [0, 0], [0, 1], [1, 0], [1, 1],
    [0, 0], [1, 1], [0, 1], [1, 0]
]
# Target: 0 = no, 1 = yes
y = [0, 0, 1, 1, 0, 1, 0, 1]

# Entropy criterion uses information gain for splitting
model = DecisionTreeClassifier(criterion="entropy", random_state=0)
model.fit(X, y)

# Predict for a new example
print("Prediction (0 = no, 1 = yes):", model.predict([[1, 0]])[0])

# Visualize the trained tree
plt.figure(figsize=(9, 5))
plot_tree(
    model,
    feature_names=["Outlook", "Temperature"],
    class_names=["No", "Yes"],
    filled=True,
    rounded=True
)
plt.title("Decision Tree using Entropy")
plt.tight_layout()
plt.show()
```

Experiment 6 — Support Vector Machine (SVM)

```
# Experiment 6: Support Vector Machine classification
# Install if required: python -m pip install scikit-learn

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

# Two input features and two classes
X = [
    [1, 1], [2, 1], [1, 2],
    [4, 4], [5, 4], [4, 5]
]
y = [0, 0, 0, 1, 1, 1]

# Create and train a linear SVM
model = SVC(kernel="linear")
model.fit(X, y)

# Predict classes for new data
X_new = [[2, 2], [5, 5]]
predictions = model.predict(X_new)

print("Predictions:", predictions)
print("Training accuracy:", accuracy_score(y, model.predict(X)))
print("Support vectors:")
print(model.support_vectors_)
print("Confusion matrix on training data:")
print(confusion_matrix(y, model.predict(X)))
```

Exp 1 : KNN

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# User-defined data: [study hours, attendance %]
X = np.array([
   [1, 40], [2, 45], [2, 50], [3, 55],
   [5, 75], [6, 80], [7, 85], [8, 90]
])

# 0 = Fail, 1 = Pass
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

# Create model
model = KNeighborsClassifier(n_neighbors=3)

# Train model
model.fit(X, y)

# Predict for a new student
new_student = np.array([[6, 78]])
prediction = model.predict(new_student)

print("Prediction:", "Pass" if prediction[0] == 1 else "Fail")
print("Training Accuracy:", accuracy_score(y, model.predict(X)))
```